

Exploratory Data Analysis on the Gold-Layer Warehouse

A walkthrough of the DataWarehouseAnalytics SQL script — from database discovery to trend analysis — explained step by step for interview-ready understanding.

T-SQL

Star Schema

gold.fact_sales

gold.dim_customers

gold.dim_products

Window Functions

✦ ACKNOWLEDGEMENT ✦

Special thanks to Eng. [Baraa Khatib Salkini](#) for the dataset design and teaching materials this practice project is built on.

✦ Before anything else — a sincere thank you to Eng. **Baraa Khatib Salkini** for the guidance behind this Data Warehouse Analytics project. This document is a personal study companion built while practicing on his star-schema dataset.

1 Measures vs. Dimensions — The Two Building Blocks of Analytics

Almost every analytical query in this script boils down to answering: **"How much of something (a measure), broken down by what (a dimension)?"** Getting this distinction clear makes every GROUP BY in the script instantly readable.

Dimensions	Measures
Descriptive, categorical attributes — the "who / what / where / when"	Numeric, quantifiable facts — the "how much / how many"
Used to slice, filter, and group data	Used inside aggregate functions: SUM, AVG, COUNT, MIN, MAX
Live in dimension tables: <code>dim_customers</code> , <code>dim_products</code>	Live in the fact table: <code>fact_sales</code>
Usually text or dates: <code>country</code> , <code>category</code> , <code>gender</code> , <code>order_date</code>	Usually numbers: <code>sales_amount</code> , <code>quantity</code> , <code>price</code>
Don't get summed — they get grouped by	Rarely appear in GROUP BY; they get aggregated

Dimensions in this project

- `country` , `gender` — from `dim_customers`
- `category` , `subcategory` , `product_name` — from `dim_products`
- `order_date` (and its YEAR/MONTH parts) — from `fact_sales`

Measures in this project

- `sales_amount` , `quantity` , `price`
- `cost` — from `dim_products`
- Derived counts: `COUNT(order_number)` , `COUNT(customer_key)`

Rule of thumb: if a column answers "which one / when / where", it's a dimension and belongs in GROUP BY. If it answers "how much / how many" and can sensibly be added up, it's a measure and belongs inside an aggregate function like SUM() or AVG().

gold.fact_sales (facts / measures)

- `order_number`, `order_date`
- `customer_key`, `product_key` (foreign keys)
- `sales_amount`, `quantity`, `price`

gold.dim_customers / gold.dim_products (dimensions)

- `customer_key`, `first_name`, `last_name`, `country`, `gender`, `birthdate`
- `product_key`, `product_name`, `category`, `subcategory`, `cost`

2 What This Script Actually Does

The script runs against a small star-schema warehouse (one fact table, two dimension tables) and works through **seven progressively deeper stages of EDA**, moving from "what tables exist" all the way to "how does revenue trend over time". Each stage below mirrors a section in the script.

01 · DATABASE EXPLORATION

Understanding what you're working with

Goal: get oriented before writing any analysis — see which tables and columns actually exist.

- Queries `INFORMATION_SCHEMA.TABLES` and `INFORMATION_SCHEMA.COLUMNS` — SQL Server's built-in metadata views — to list every table/view and every column without needing to open the database diagram manually.

02 · DIMENSIONS EXPLORATION

Mapping the categorical landscape

Goal: discover the distinct categories that data can be grouped by, before aggregating anything.

- `SELECT DISTINCT country FROM dim_customers` — which countries customers come from.
- `SELECT DISTINCT category, subcategory, product_name` — the full product hierarchy, from broad category down to individual product.

03 · DATE EXPLORATION

Finding the boundaries of the data

Goal: understand the timespan covered and the age range of customers, using `MIN/MAX` and `DATEDIFF`.

- First vs. last `order_date`, plus the span expressed in years, months, and days — useful for knowing whether you're analyzing a full year of data or just a slice.
- Oldest and youngest customer `birthdate`, converted to an age using `DATEDIFF(YEAR, birthdate, GETDATE())`.

04 · MEASURES EXPLORATION

The business's big numbers

Goal: calculate the core KPIs at the highest level of aggregation — no grouping yet, just single totals.

- Total sales (`SUM(sales_amount)`), total quantity sold, average selling price, total orders (both raw and distinct), total products, and total customers.
- These individual queries are then stitched together with `UNION ALL` into a single "report" result set — one row per KPI, labeled with a `measure_name` column. This is a simple but effective pattern for building a one-glance business summary.

05 · MAGNITUDE ANALYSIS

Measures broken down by dimensions

Goal: this is where measures and dimensions finally meet — comparing a measure's size across different categories.

- Customers grouped by country and by gender (`COUNT` as the measure).
- Products grouped by category, with counts and average cost.
- Revenue by category and by customer — these require joining `fact_sales` to the dimension tables first, since the measure (`sales_amount`) lives in the fact table but the grouping label (category, customer name) lives in the dimensions.

Note in the script: the fact-to-dimension joins intentionally use `LEFT JOIN` (fact table first) rather than `INNER JOIN`, so any sales row without a matching product or customer record is still kept in the totals instead of being silently dropped.

06 · RANKING ANALYSIS

Top and bottom performers

Goal: identify the best and worst performing dimension values for a given measure.

- `SELECT TOP 5 ... ORDER BY total_revenue DESC` for best-selling products, and `ASC` for the worst-performing ones.
- The same "top 5 products" question is repeated using `ROW_NUMBER() OVER (ORDER BY SUM(sales_amount) DESC)` inside a subquery, filtered with `WHERE rank_products <= 5` — a window-function alternative to `TOP`, useful when ties need explicit handling or ranks need to be reused downstream.
- Top 10 customers by revenue, and the 3 customers with the *fewest* distinct orders (correctly sorted `ASC` on `total_orders` — the script notes this was originally written as `DESC` by mistake, which would have returned the most active customers instead of the least active ones).

Goal: track a measure's evolution across time to spot seasonality, growth, or decline.

- Monthly totals using `YEAR(order_date) / MONTH(order_date)`, then the same result reproduced more cleanly with `DATETRUNC(MONTH, order_date)` (SQL Server 2022+), which collapses each date to the first of its month in a single column instead of two.
- A CTE (`monthly_sales`) feeds a final query that layers on two window functions over the monthly totals: a **running total** (`SUM( . . . ) OVER (ORDER BY order_month)`) and a **month-over-month comparison** using `LAG( )` to pull the previous month's value and subtract it from the current one.



Key SQL Techniques Practiced

- **Aggregate functions:** `SUM`, `AVG`, `COUNT`, `COUNT(DISTINCT)`, `MIN`, `MAX`
- **Joins:** `LEFT JOIN` from fact to dimension tables (deliberately, to avoid dropping unmatched fact rows)
- **Window functions:** `ROW_NUMBER()`, `SUM() OVER()`, `LAG()`
- **Date functions:** `DATEDIFF`, `YEAR()`, `MONTH()`, `DATETRUNC()`
- **CTEs:** structuring the monthly trend query for readability before applying window functions on top
- **UNION ALL:** combining several single-value KPI queries into one business-metrics report
- **TOP N / ranking patterns:** both the simple `TOP` clause and the more flexible window-function ranking approach

Generated as a study companion for the DataWarehouseAnalytics SQL EDA script · star schema: `gold.fact_sales`, `gold.dim_customers`, `gold.dim_products`.